



GUÍA DE DESARROLLO - LEXLY SAAS

De Cero a Producción Construyendo un SaaS de Gestión Jurídica

Versión: 1.0

Proyecto: Lexly - SaaS de Gestión Jurídica

Stack: FastAPI + PostgreSQL + React + TypeScript

Fecha: Mayo 2026



ÍNDICE DE LA GUÍA

LEXLY - Roadmap de Desarrollo

FASE 1: FUNDAMENTOS DEL PROYECTO

- └─ 1.1 Estructura de Carpetas
- └─ 1.2 Configuración Python
- └─ 1.3 Entorno Virtual
- └─ 1.4 Dependencias Base

FASE 2: BASE DE DATOS

- └─ 2.1 PostgreSQL Local
- └─ 2.2 Esquema de Lexly
- └─ 2.3 Tablas y Relaciones
- └─ 2.4 Datos de Prueba

FASE 3: BACKEND API

- └─ 3.1 FastAPI Básico
- └─ 3.2 Modelos con Drizzle
- └─ 3.3 CRUD de Entidades
- └─ 3.4 Autenticación JWT

FASE 4: FRONTEND

- └─ 4.1 Next.js Setup
- └─ 4.2 Componentes Base
- └─ 4.3 Integración API

FASE 5: MULTI-TENANCY

- └─ 5.1 Row-Level Security
- └─ 5.2 Aislamiento
- └─ 5.3 Deployment

🎯Cómo Usar Esta Guía

Paso	Acción
1	Lee la explicación (para entender el "por qué")
2	Ejecuta el comando (para hacerlo funcionar)
3	Verifica el resultado (para confirmar que funciona)
4	продолжает al siguiente paso

No te apresures: Esta guía está diseñada para aprender. Si algo no funciona, vuelve a leer e intenta de nuevo.

FASE 1: FUNDAMENTOS DEL PROYECTO

1.1 ESTRUCTURA DE CARPETAS

📖 Explicación

Vamos a crear una estructura de proyecto profesional. Esto facilita: - Encontrar archivos rápidamente - Separar responsabilidades (modelos, rutas, servicios) - Escalar el proyecto sin caos - Trabajar en equipo

La estructura sigue el patrón **Domain-Driven Design** simplificado:

```
lexlySaas/
├── backend/           # API (Python/FastAPI)
│   ├── app/
│   │   ├── models/   # Modelos de datos
│   │   ├── routes/   # Endpoints de API
│   │   ├── schemas/  # Tipos/Pydantic
│   │   ├── services/ # Lógica de negocio
│   │   └── db/        # Conexión a BD
│   ├── migrations/   # Cambios de BD
│   └── tests/         # Pruebas
├── frontend/         # Frontend (Next.js/React)
│   ├── app/          # Páginas
│   ├── components/   # Componentes reutilizables
│   ├── lib/          # Utilidades
│   └── public/        # Archivos estáticos
└── docs/             # Documentación
```

执行命令

Ya tienes la estructura creada. Verificamos:

```
$ ls -R /media/chesdevos/CHESEVS1/proyectos/lexlySaas/
```

Deberías ver:

```
lexlySaas/  
├── backend/  
│   ├── app/  
│   └── __init__.py  
├── frontend/  
│   ├── app/  
│   ├── components/  
│   ├── lib/  
│   └── public/  
├── .git/  
├── .idea/  
├── pyproject.toml  
└── README.md
```

Verificación

Si tu estructura es похожа, ¡perfecto! Si no, ejecuta:

```
mkdir -p /media/chesdevos/CHESEVS1/proyectos/lexlySaas/backend/app/{models,routes,schema}  
mkdir -p /media/chesdevos/CHESEVS1/proyectos/lexlySaas/backend/migrations  
mkdir -p /media/chesdevos/CHESEVS1/proyectos/lexlySaas/backend/tests  
mkdir -p /media/chesdevos/CHESEVS1/proyectos/lexlySaas/docs
```

1.2 CONFIGURACIÓN PYTHON

Explicación

pyproject.toml es el archivo de configuración de tu proyecto Python moderno. Define: - Nombre del proyecto - Versión - Descripción - Dependencias - Python requerido

Esto reemplaza al antiguo `requirements.txt`.

执行命令

Edita el archivo de configuración:

```
$ cat > /media/chesdevos/CHESEVS1/proyectos/lexlySaas/pyproject.toml << 'EOF'  
[project]
```

```

name = "lexlysaas"
version = "0.1.0"
description = "SaaS de gestión jurídica para despachos de extranjería"
readme = "README.md"
requires-python = ">=3.11"
dependencies = []

[project.optional-dependencies]
dev = [
    "pytest>=8.0.0",
    "pytest-asyncio>=0.23.0",
    "pytest-cov>=4.1.0",
]

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[tool.pytest.ini_options]
testpaths = ["tests"]
python_files = ["test_*.py"]
python_functions = ["test_*"]
asyncio_mode = "auto"

[tool.ruff]
line-length = 100
target-version = "py311"

[tool.ruff.lint]
select = ["E", "F", "I", "N", "W"]
ignore = ["E501"]
EOF

```

✓ Verificación

Verifica que el archivo se creó correctamente:

```
$ cat /media/chesdevos/CHESDEV51/proyectos/lexlySaas/pyproject.toml
```

Deberías ver el contenido del archivo.

1.3 ENTORNO VIRTUAL

📖 Explicación

Un entorno virtual es como una **habitación aislada** para cada proyecto Python. ¿Por qué?

Problema sin entorno virtual: -同一个 Python para todos los proyectos - Conflicto de versiones (Proyecto A necesita Python 3.10, Proyecto B 3.12) - Diferentes librerías incompatibles

Solución: Entorno virtual - Cada proyecto tiene su propio Python - Las librerías no chocan entre proyectos - Puedes trabajar en muchos proyectos simultáneamente

执行命令

Crea el entorno virtual:

```
# En la carpeta del proyecto
cd /media/chesdevos/CHESEVS1/proyectos/lexlySaas

# Crear entorno virtual (Python 3.11+)
python3 -m venv .venv

# Activar el entorno
source .venv/bin/activate # Linux/Mac
# .venv\Scripts\activate # Windows
```

Si ves **(.venv)** al principio de tu línea de comandos, ¡estás dentro!

```
(.venv) user@machine:~/lexlySaas $
```

Verificación

```
# Verifica que estás en el entorno
which python

# Debería mostrar algo como:
# /media/chesdevos/CHESEVS1/proyectos/lexlySaas/.venv/bin/python
```

1.4 DEPENDENCIAS BASE

Explicación

Ahora instalamos las **dependenciasbase** del proyecto:

Librería	Propósito	¿Por qué?
fastapi	Framework web	API moderna y rápida
uvicorn	Servidor ASGI	Ejecuta FastAPI

Librería	Propósito	¿Por qué?
pydantic	Validación	Tipos seguros
sqlalchemy	ORM	Base de datos
psycopg2-binary	Driver PostgreSQL	Conexión a BD
python-jose	JWT	Autenticación
passlib	Hash de contraseñas	Seguridad

执行命令

Instala las dependencias:

```
# En tu entorno virtual activado
pip install fastapi uvicorn pydantic sqlalchemy psycopg2-binary python-jose[cryptography]

# Instalar como desarrollo (opcional)
pip install pytest pytest-asyncio pytest-cov
```

Verificación

```
$ python -c "import fastapi; print(fastapi.__version__)"
# Debería mostrar: 0.115.0 (o similar)
```

FASE 2: BASE DE DATOS

2.1 POSTGRESQL LOCAL

Explicación

PostgreSQL es nuestra base de datos. Para desarrollo local, tienes opciones:

Opción	Difficulty	Uso
Docker	Easy	Rápido, portable
Install local	Medium	Instalación nativa
Supabase local	Easy	Emula Supabase

Usaremos **Docker** por simplicidad.

执行命令

Inicia PostgreSQL con Docker:

```
# Crear y ejecutar contenedor PostgreSQL
docker run -d \
  --name lexly-postgres \
  -e POSTGRES_PASSWORD=lexly2026 \
  -e POSTGRES_DB=lexly \
  -e POSTGRES_USER=postgres \
  -p 5432:5432 \
  postgres:15-alpine

# Verificar que está corriendo
docker ps | grep lexly-postgres
```

Credenciales configuradas: - Usuario: `postgres` - Contraseña: `lexly2026` - Base de datos: `lexly` - Puerto: `5432`

Verificación

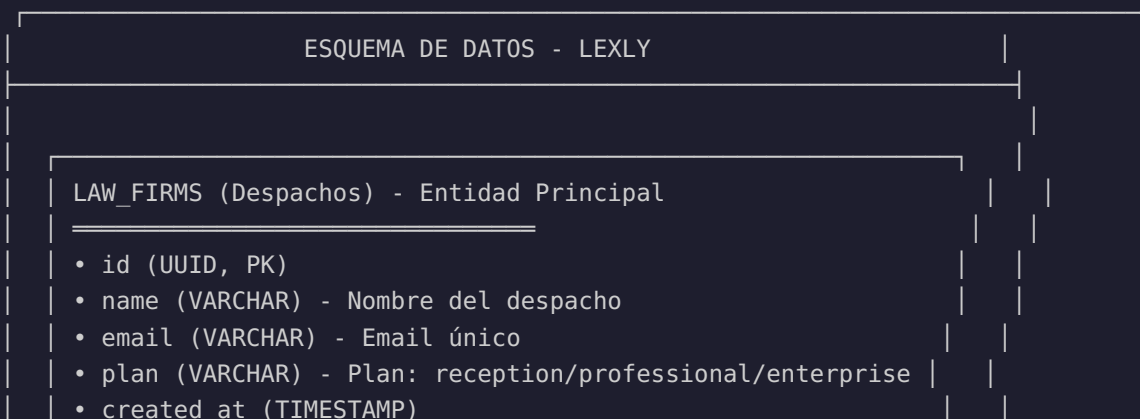
```
# Conectar a PostgreSQL
docker exec -it lexly-postgres psql -U postgres -c "SELECT version();"

# Debería mostrar la versión de PostgreSQL
```

2.2 ESQUEMA DE LEXLY

Explicación

Diseño de la base de datos según el documento de diseño. Aquí están las **entidades principales**:



| 1:N



USERS (Usuarios del sistema)

- id (UUID, PK)
- law_firm_id (UUID, FK) -> law_firms
- email (VARCHAR) - Email único
- password_hash (VARCHAR)
- full_name (VARCHAR)
- role (VARCHAR) - admin/lawyer/assistant
- is_active (BOOLEAN)

| 1:N



CLIENTS (Clientes/Personas atendidas)

- id (UUID, PK)
- law_firm_id (UUID, FK) -> law_firms
- first_name (VARCHAR)
- last_name (VARCHAR)
- email (VARCHAR)
- phone (VARCHAR) - Teléfono
- nationality (VARCHAR)
- nie_passport (VARCHAR)
- is_active (BOOLEAN)
- created_at (TIMESTAMP)

000

| 1:N



CASES (Casos/Expedientes)

- id (UUID, PK)
- client_id (UUID, FK) -> clients
- assigned_to (UUID, FK) -> users
- case_type (VARCHAR) - arraigo_laboral/renovacion/etc
- status (VARCHAR) - nuevo/documentacion/presentado/etc
- title (VARCHAR)
- description (TEXT)
- deadline (DATE)
- priority (VARCHAR) - normal/high/urgent
- created_at (TIMESTAMP)

| 1:N



DOCUMENTS (Documentos)

- id (UUID, PK)
- case_id (UUID, FK) -> cases
- client_id (UUID, FK) -> clients
- name (VARCHAR) - Nombre del archivo
- file_path (TEXT) - Ruta en Storage

- doc_type (VARCHAR) - DNI/expediente/etc
- created_at (TIMESTAMP)

ACTIVITIES (Historial)

- id (UUID, PK)
- client_id (UUID, FK) -> clients
- case_id (UUID, FK) -> cases
- user_id (UUID, FK) -> users
- activity_type (VARCHAR)
- content (TEXT)
- created_at (TIMESTAMP)

执行命令

Conecta a PostgreSQL y crea el esquema:

```
# Conectar usando psql
docker exec -it lexly-postgres psql -U postgres -d lexly
```

Ahora ejecuta el SQL para crear las tablas:

```
-- =====
-- LEXLY - ESQUEMA DE BASE DE DATOS
-- PostgreSQL 15+ / Supabase
-- =====

-- Extensiones necesarias
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- =====
-- 1. LAW_FIRMS (Despachos/Multi-tenant)
-- =====
CREATE TABLE law_firms (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  phone VARCHAR(50),
  address TEXT,
  plan VARCHAR(50) DEFAULT 'recepcion'
  CHECK (plan IN ('recepcion', 'professional', 'enterprise')),
  has_ia BOOLEAN DEFAULT FALSE,
  max_users INTEGER DEFAULT 5,
  max_clients INTEGER DEFAULT 500,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

```

-- =====
-- 2. USERS (Usuarios)
-- =====
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    law_firm_id UUID REFERENCES law_firms(id) ON DELETE CASCADE,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(255) NOT NULL,
    role VARCHAR(50) DEFAULT 'assistant'
        CHECK (role IN ('admin', 'lawyer', 'assistant')),
    avatar_url TEXT,
    is_active BOOLEAN DEFAULT TRUE,
    last_login_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Índices para users
CREATE INDEX idx_users_law_firm ON users(law_firm_id);
CREATE INDEX idx_users_email ON users(email);

-- =====
-- 3. CLIENTS (Clientes)
-- =====
CREATE TABLE clients (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    law_firm_id UUID REFERENCES law_firms(id) ON DELETE CASCADE,

    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    email VARCHAR(255),
    phone VARCHAR(50) NOT NULL,
    nationality VARCHAR(100),
    nie_passport VARCHAR(50),
    date_of_birth DATE,
    country_origin VARCHAR(100),
    immigration_status VARCHAR(100),
    address TEXT,
    emergency_contact VARCHAR(255),
    metadata JSONB DEFAULT '{} '::jsonb,
    tags TEXT[],
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Índices para clientes
CREATE INDEX idx_clients_law_firm ON clients(law_firm_id);
CREATE INDEX idx_clients_name ON clients(last_name, first_name);
CREATE INDEX idx_clients_phone ON clients(phone);
CREATE INDEX idx_clients_active ON clients(is_active) WHERE is_active = TRUE;

-- =====
-- 4. CASES (Casos/Expedientes)
-- =====

```

```

CREATE TABLE cases (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  client_id UUID REFERENCES clients(id) ON DELETE CASCADE,
  assigned_to UUID REFERENCES users(id) ON DELETE SET NULL,

  case_type VARCHAR(100) NOT NULL,
  status VARCHAR(50) DEFAULT 'nuevo',
  title VARCHAR(255) NOT NULL,
  description TEXT,
  application_date DATE,
  deadline DATE,
  resolution_date DATE,
  priority VARCHAR(20) DEFAULT 'normal',
  metadata JSONB DEFAULT '{}'::jsonb,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),

  CONSTRAINT cases_type CHECK (case_type IN (
    'arraigo_laboral', 'arraigo_social', 'arraigo_familiar',
    'renovacion', 'modificacion', 'ciudadania',
    'reagrupacion', 'asilo', 'otros'
  )),
  CONSTRAINT cases_status CHECK (status IN (
    'nuevo', 'documentacion', 'presentado',
    'en_proceso', 'resuelto', 'archivado'
  ))
);

-- Índices para cases
CREATE INDEX idx_cases_client ON cases(client_id);
CREATE INDEX idx_cases_assigned ON cases(assigned_to);
CREATE INDEX idx_cases_status ON cases(status);
CREATE INDEX idx_cases_deadline ON cases(deadline);

-- =====
-- 5. DOCUMENTS (Documentos)
-- =====

CREATE TABLE documents (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  case_id UUID REFERENCES cases(id) ON DELETE CASCADE,
  client_id UUID REFERENCES clients(id) ON DELETE CASCADE,
  uploaded_by UUID REFERENCES users(id) ON DELETE SET NULL,
  name VARCHAR(255) NOT NULL,
  doc_type VARCHAR(100),
  file_path TEXT NOT NULL,
  file_size BIGINT,
  mime_type VARCHAR(100),
  is_verified BOOLEAN DEFAULT FALSE,
  verified_by UUID REFERENCES users(id),
  verified_at TIMESTAMPTZ,
  metadata JSONB DEFAULT '{}'::jsonb,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Índices para documents
CREATE INDEX idx_documents_case ON documents(case_id);

```

```

CREATE INDEX idx_documents_client ON documents(client_id);

-- =====
-- 6. ACTIVITIES (Historial)
-- =====
CREATE TABLE activities (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  client_id UUID REFERENCES clients(id) ON DELETE CASCADE,
  case_id UUID REFERENCES cases(id) ON DELETE SET NULL,
  user_id UUID REFERENCES users(id) ON DELETE SET NULL,
  activity_type VARCHAR(50) NOT NULL,
  content TEXT NOT NULL,
  metadata JSONB DEFAULT '{}'::jsonb,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Índices para activities
CREATE INDEX idx_activities_client ON activities(client_id);
CREATE INDEX idx_activities_case ON activities(case_id);
CREATE INDEX idx_activities_user ON activities(user_id);

```

✓ Verificación

```

-- Verificar tablas creadas
\dt

--, 你应该看到:
--      List of relations
-- Schema | Name      | Type  | Owner
-- -----+-----+-----+-----
-- public | activities | table | postgres
-- public | cases      | table | postgres
-- public | clients    | table | postgres
-- public | documents  | table | postgres
-- public | law_firms  | table | postgres
-- public | users      | table | postgres
-- (6 rows)

```

2.3 AÑADIR CLAVES LLAVE (KEYS)

📖 Explicación

Para que actualice automáticamente el campo `updated_at` cuando modificamos un registro, usamos **triggers**.


```
-- Nota: En un proyecto real, el password_hash se genera con passlib:
-- >>> from passlib.hash import bcrypt
-- >>> bcrypt.hash("password123")
```



```
--, 你应该 ver:
--  table_name | total
--  -----+-----
--  law_firms  |      3
--  users      |      3
-- (2 rows)
```

FASE 3: BACKEND API

(Continuará en la siguiente parte de la guía...)

Concepto	Descripción
Entorno virtual	Aislamiento de proyectos Python
pyproject.toml	Configuración moderna de Python
PostgreSQL + Docker	Base de datos en contenedor
Esquema de BD	Estructura de tablas para Lexly
Triggers	automatización de timestamps
Seed data	Datos de prueba



PRÓXIMO: BACKEND CON FASTAPI

En la siguiente parte de la guía, aprenderemos: 1. Configurar la conexión a la base de datos 2. Crear el servidor FastAPI básico 3. Crear el primer endpoint (Health Check) 4. CRUD completo de entidades

Recordá: El objetivo es aprender haciendo. Si algo no funciona, volvé a leer e intentá de nuevo.

¡Éxito en tu aprendizaje! 🚀